

Sistema de Reconocimiento Automático de Matrículas

1. Contexto del Proyecto

En esta tarea construirás en parejas, un proyecto que reciba la imagen de un coche y devuelva el texto de su matrícula utilizando modelos de IA.

2. Objetivos de Aprendizaje

- Implementar un modelo de detección de objetos **YOLO** (v8, v10 o v11) para localizar matrículas.
- Manipular imágenes con **OpenCV** (recorte y preprocesamiento).
- Integrar una librería de **OCR** (EasyOCR o PaddleOCR) para la extracción de texto.
- Gestionar el flujo de datos entre diferentes modelos de IA.

3. Requisitos Técnicos (Stack)

- **Lenguaje:** Python
 - **Detección:** `ultralytics` (YOLOv8 o YOLOv11).
 - **OCR:**
 - `easyocr` (muy sencillo de usar)
`pip install easyocr`
 - `paddleocr` (más rápido y preciso en entornos industriales).
`pip install paddleocr`
 - **Procesamiento:** `opencv-python` y `numpy`.
 - **Entorno:** Jupyter Notebook
-

4. Instrucciones

Fase 1: Detección de la Matrícula

1. **Carga del modelo:** Utiliza un modelo YOLO pre-entrenado de Hugging Face o bien entrena el tuyo con un dataset propio o de Roboflow.

Específico para matrículas de Hugging Face:

<https://huggingface.co/models?search=license+plate+yolo>

<https://huggingface.co/models?search=matriculas+yolo>

2. **Inferencia:** Pasa una imagen de un coche al modelo y obtén las coordenadas (`bounding box`) de la matrícula.
3. **Filtrado:** Asegúrate de seleccionar solo la detección con mayor confianza (`conf > 0.5`).

Fase 2: Recorte y Preprocesamiento

1. **Recorte:** Utiliza las coordenadas obtenidas para recortar la zona de la matrícula de la imagen original usando OpenCV.
2. **Preprocesa:** Convierte el recorte a escala de grises y aplica un umbral (`thresholding`) para que el texto resalte sobre el fondo. Esto mejora la precisión del OCR.

Fase 3: Lectura con OCR

1. **Instanciación:** Configura el lector de OCR para el idioma local (ej: `reader = easyocr.Reader(['es'])`).
2. **Extracción:** Pasa la imagen recortada al OCR.
3. **Post-procesamiento:** Limpia el texto resultante (quitar espacios, caracteres especiales) y valida que cumpla el formato de matrícula de tu país (4 números y 3 letras).

Fase 4: Visualización

1. Dibuja el cuadro sobre la imagen original.
2. Escribe el texto detectado justo encima del cuadro usando `cv2.putText`.

5. Entregables

1. **Código fuente:** Un archivo `.ipynb` bien documentado.
 2. **Carpeta de resultados:** Al menos 5 imágenes de prueba donde se vea el coche original y el resultado final con el texto superpuesto.
 3. **Breve informe:** Comparativa de resultados (¿Qué pasa si la matrícula está inclinada o hay poca luz?).
-

Extra

- **Tracking:** Si usas un video, utiliza `model.track()` para seguir la misma matrícula durante varios frames y evitar lecturas duplicadas.
- **Detección de Coche + Matrícula:** Haz un modelo que primero detecte "coche" y luego busque la "matrícula" solo dentro de esa zona para evitar leer carteles de la calle.
- **Exportación:** Guarda los resultados (matrícula + hora) en un archivo CSV